INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Construcción de software				
TÍTULO DE LA PRÁCTICA:	*Diseñar la arquitectura guiada por atributos de calidad: árbol de utilidad, ADD, tácticas, evaluación ATAM-lite con IA y fitness functions automatizadas*				
NÚMERO DE PRÁCTICA:	*08*	**AÑO LECTIVO:**	*2026B*	**NRO. SEMESTRE:**	*VI*
FECHA DE PRESENTACIÓN	*31/10/26*	**HORA DE PRESENTACIÓN**			
INTEGRANTE (s): Oliva Valdivia David Alexander				**NOTA:**	
DOCENTE(s): *Mg. Antonio Arroyo Paz*					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Repositorio: `github.com/DavidOliva06/cs-2026b-lab04-grupo06`. Caso: **SismoReporta AQP**. Escenario de crecimiento: **simulacro nacional**, con 20 000 reportes en 1 hora, una ráfaga de 2 000 en 1 minuto al volver la señal y la red móvil congestionada; el sistema pasa de Arequipa a las 25 regiones. Atributo crítico: **disponibilidad**.

La entrega del 31/10/26 terminó en el commit `0b802c3` sin la etiqueta de entrega, con el workflow y `tests/test_fitness.py` vacíos. Esta versión del informe corresponde a la etiqueta `v0.8-arquitectura` (commit `05593f1`, rama `lab08-correcciones`), que completa E5 y E8 (ver "Revisión de la entrega" al final de esta sección).

Tabla 1. Commits que respaldan cada ejercicio.

Ejercicio	Commit	Mensaje	Fecha
E1	`a4012b2`, `9a28bdb`	E1: arbol de utilidad (el segundo crea las carpetas y archivos vacíos de E2–E8)	2026-10-31
E2	`a0f35d7`	E2: Iteracion 1 ADD	2026-10-31
E3	—	No realizado (trabajo individual, sin grupo asignado)	—
E4	`f514a26`	E4: Iteraciones 2 y 3	2026-10-31
E5	`c234e7f`	E5: Fitness (contratos de import-linter y separación de `entrada.py`)	2026-10-31
E6, E7 y E8	`0b802c3`	E8: README (incluye ADR-006 a ADR-008, C4, atam-lite y bitácora)	2026-10-31

E5 y E8 (revisión)	90a3b13, 3fd719e, 117b86b, 05593f1	Fitness de calidad y workflow; violación de prueba y su reversión; README e insignia (etiqueta v0.8-arquitectura)	2026-11-13

E1: Árbol de utilidad

1. **Parta de drivers.md (Lab 04) y del escenario de crecimiento de su caso. Use el Prompt IA 1 para obtener escenarios candidatos.**

Prompt usado (bitácora, interacción 7), con el contexto del caso:

```
Actúa como arquitecto de software. Contexto: drivers.md, ADR-001 y el nuevo contexto: simulacro
nacional, 20 000 reportes en 1 hora, ráfaga de 2 000 en 1 min al volver la señal, red móvil
congestionada y 25 regiones. Tarea: propón escenarios de calidad de seis partes (fuente, estímulo,
entorno, artefacto, respuesta, medida) para rendimiento, disponibilidad, seguridad y
modificabilidad. Para cada uno sugiere (importancia, dificultad) con H/M/L y justifícalo.
Restricciones: medidas numéricas; no propongas soluciones todavía.
```

El primer contexto que se pegó en el prompt era el ejemplo de la guía (de otro caso). La IA lo detectó al compararlo con `drivers.md` y se reemplazó por el del simulacro nacional; queda registrado en la bitácora. La IA devolvió 13 escenarios candidatos (D1–D4, R1–R3, S1–S3, M1–M3), en `docs/architecture/utilidad/escenarios.md`.

2. **Seleccione 6 o más escenarios que cubran los 4 atributos (rendimiento, disponibilidad, seguridad y modificabilidad), cada uno con medida numérica.**

Tabla 2. Los 10 escenarios del árbol (respuesta y medida numérica).

ID	Atributo	Estímulo	Medida
D1 (QA-01)	Disponibilidad	20 000 reportes en 60 min, ráfaga de 2 000 en 1 min, red congestionada	≥ 99,5 % guardados ≤ 15 min tras recuperar la señal; 0 duplicados; 0 perdidos en el dispositivo
D2	Disponibilidad	El proceso de la API termina en pleno pico	Recepción restablecida ≤ 60 s; 0 confirmados perdidos; ingesta disponible ≥ 98 % de la hora
D3	Disponibilidad	El volumen de fotos llega al 95 %	100 % de los reportes sin foto aceptados; 0 s de caída de PostgreSQL; aviso ≤ 60 s
D4	Disponibilidad	5 000 PWA reintentan tras 10 min de caída	0 caídas nuevas en 30 min; pendientes guardados ≤ 10 min; ≤ 1 % de 5xx
R1	Rendimiento	Ráfaga de 33 reportes por s	Confirmación p95 ≤ 2 s, p99 ≤ 5 s; ≥ 35 reportes/s durante 5 min
R2 (QA-02)	Rendimiento	30 coordinadores refrescan el mapa cada 30 s con 20 000 reportes	Carga del mapa p95 ≤ 3 s; filtro p95 ≤ 1 s; retraso ≤ 60 s
S1 (QA-03)	Seguridad	50 intentos de ver fotos y ubicaciones ajenas	100 % rechazados (403) y registrados; 0 bytes devueltos; TLS ≥ 1.2
S2	Seguridad	1 000 reportes falsos en 5 min desde un dispositivo	≤ 10 aceptados en 10 min; 0 zonas alteradas; ≥ 99,5 % legítimos aceptados; aviso ≤ 5 min
M1	Modificabilidad	Extender la cobertura a las 25 regiones	≤ 3 días-persona; 0 archivos del módulo de reportes; ≤ 5 archivos en total
M2	Modificabilidad	Nuevo criterio de priorización (colegios, hospitales)	≤ 1 día-persona; 0 archivos fuera de priorización; ≤ 60 s sin ingesta

Quedan fuera del árbol R3 (priorización automática fuera del MVP), S3 (M,L; no se prueba en el simulacro) y M3 (L,L; ADR-005 ya lo cubre). Ninguna medida dice "rápido", "seguro" o "fácil".

3. **Califique cada escenario con (importancia, dificultad) y corrija al menos una calificación de la IA,**

justificando el cambio.

Tabla 3. Calificaciones: IA frente a final.

ID	IA	Final	ID	IA	Final
D1	(H,H)	(H,H)	R2	(H,M)	**(H,H)**
D2	(H,M)	(H,M)	S1	(H,L)	(H,L)
D3	(H,M)	(H,M)	S2	(H,M)	(H,M)
D4	(H,H)	(H,H)	M1	(H,M)	(H,M)
R1	(H,M)	(H,M)	M2	(M,L)	(M,L)

Corrección de R2, de (H,M) a (H,H). La IA calificó la dificultad como M porque el mapa "es una consulta de lectura". Esa justificación ignora dos consecuencias de ADR-001: el servidor es único (R-03) y la consulta corre sobre la misma PostgreSQL que, en la ráfaga de R1, recibe unas 33 escrituras por segundo; y el panel no se puede escalar por separado. Además, R2 pide 4 veces los reportes de QA-02 (20 000 frente a 5 000) y 3 veces los coordinadores (30 frente a 10) con el mismo p95 ≤ 3 s. Hay al menos un (H,H) o (H,M) por atributo: D1 y D4 (H,H), R2 (H,H), S2 (H,M) y M1 (H,M).

4. Escriba el árbol en Graphviz (arbol.dot, como en la Figura 3), renderícelo y haga commit con el mensaje E1: arbol de utilidad.

```
digraph utilidad {          // docs/architecture/utilidad/arbol.dot (extracto)
  rankdir=LR;
  Utilidad -> Disponibilidad -> "Ingesta bajo pico" -> D1;
  Utilidad -> Rendimiento    -> "Consulta del mapa" -> R2;
  Utilidad -> Seguridad      -> "Integridad de los reportes" -> S2;
  Utilidad -> Modificabilidad -> "Ampliación de cobertura" -> M1;
  D1 [label="(H,H) D1: 20 000 reportes en 60 min ... -> >= 99,5 % guardados <= 15 min ..."];
}
```

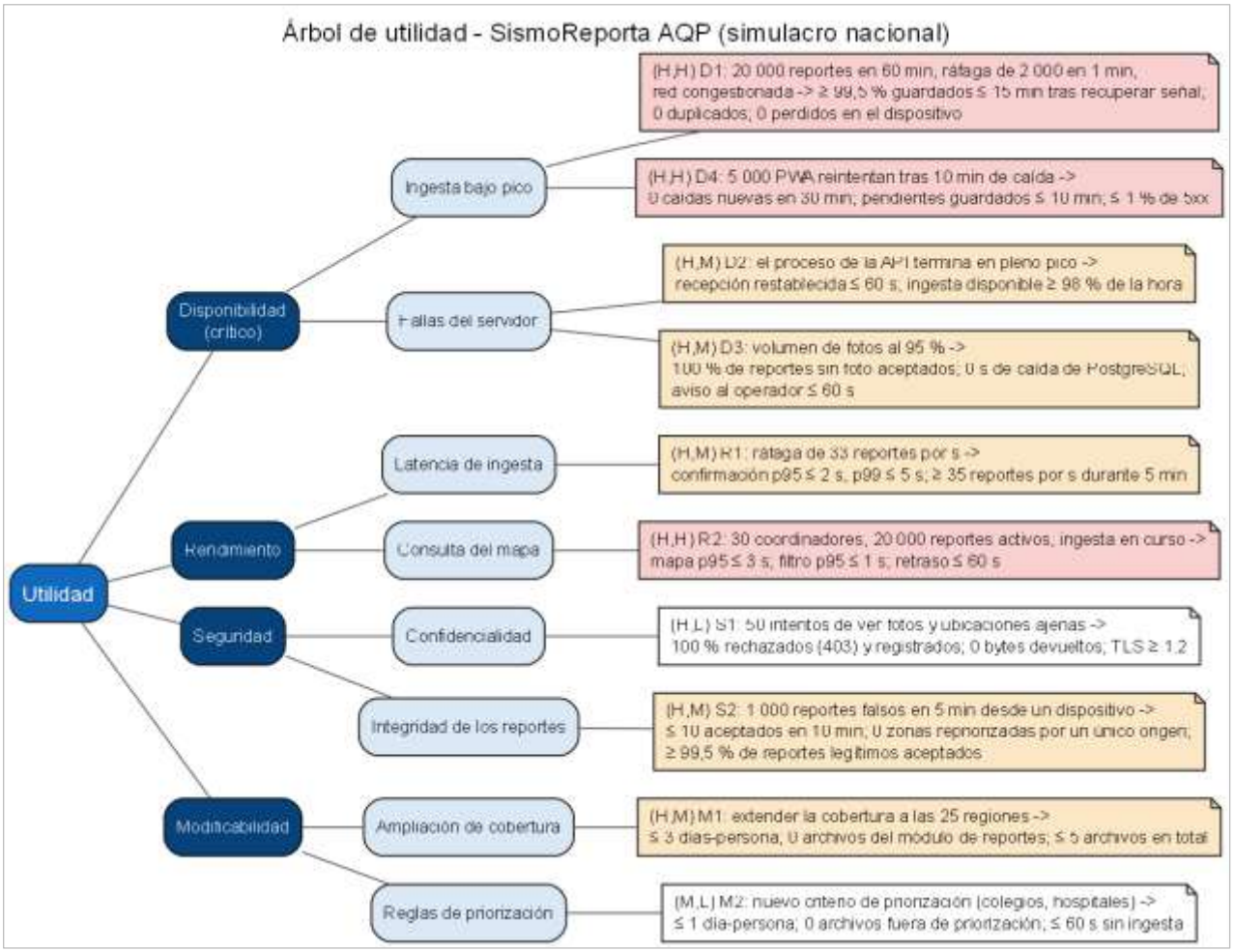


Figura 1. Árbol de utilidad (dot -Tpng arbol.dot -o arbol.png). Rojo: (H,H); naranja: (H,M); blanco: resto.

Commit a4012b2 "E1: arbol de utilidad".

E2: Iteración 1 de ADD con evidencia

1. Elija como objetivo el escenario de mayor prioridad y dificultad.

Objetivo: **D1** (H,H). Hay tres (H,H): D1, D4 y R2. Se elige D1 porque la disponibilidad es el atributo crítico y D1 extiende QA-01; D4 es un caso de recuperación de D1 y R2 pertenece al atributo n.º 2. Elementos refinados: PWA (cola local y sincronizador), proxy inverso, API de ingesta, worker y almacenamiento de fotos.

2. Con el Prompt IA 2, obtenga 3 o más conceptos de diseño candidatos; evalúe sus trade-offs y rechace al menos uno con argumentos.

Tabla 4. Conceptos de diseño evaluados para D1 (add/iteracion-1.md, secciones 5 a 8).

Táctica (Bass et al.)	Qué es en este sistema	Decisión	Argumento
T2. Degradación (modo pico)	Sobre el 70 % de conexiones, el servidor acepta solo los datos, la PWA guarda la foto para después y el worker solo prioriza	**Aceptada**	Única que cumple D1 con una caída de 10 min (de 97,47 % a 100 %); en la base baja el máximo de 298 s a 18 s
T1. Limitar la respuesta a eventos	HTTP 429 con `Retry-After` sobre 35 reportes/s	Rechazada	No mejora D1 en ninguna variante; con 1 núcleo y caída, T1 + T2 no cumple (99,28 %) y solo T2 sí (99,86 %)
T3. Aumentar recursos temporalmente	Agrandar el servidor 24–48 h alrededor del simulacro	**Rechazada**	Invalida el simulacro como medición: en un sismo real nadie agranda el servidor antes; y si se olvida bajar el plan, el costo se multiplica

			por 3 o 4 (R-03)
T4. Redundancia pasiva	Segundo servidor con réplica de PostgreSQL	Rechazada	US$ 40–60 al mes, fuera de R-03; no ataca la causa (fotos y conexiones lentas)

La IA recomendó primero T2 + T1 con T3 "como margen"; al pedirle que reanalizara, la propia IA rechazó T3, y la evidencia de la simulación rechazó después T1. **Se mantiene el estilo de ADR-001.**

3. **Genere evidencia: adapte una de las simulaciones del ejemplo con datos de su caso (tasa de llegada, tiempos de servicio, probabilidad de falla) o construya un prototipo mínimo. Debe mostrar el escenario antes y después de la táctica.**

Simulación de eventos discretos `fitness/simulacion_d1.py` (solo biblioteca estándar), con la misma medida de D1: % de reportes guardados ≤ 15 min después de recuperar la señal.

Tabla 5. Supuestos de la simulación (add/iteracion-1.md, sección 9.2).

Parámetro	Valor
Tasa de llegada	18 000 reportes uniformes en 60 min + 2 000 entre los minutos 20 y 21
Red	Latencia lognormal (mediana 1,5 s); subida lognormal (mediana 50 kbps); 0,01 cortes por s de conexión
Tamaños	Datos 1 KB; foto 300 KB
PWA (ADR-003)	Espera 30 s la respuesta; reintentos aleatorios entre 0 y min(120 s, 2 s × 2^n)
Servidor (ADR-001, R-03)	1 000 conexiones en el proxy; 2 núcleos compartidos por la API y 1 worker (corregido a 1 núcleo en 9.7)
Tiempos de servicio	Datos 30 ms; duplicado 10 ms; guardar foto 20 ms; procesar foto 150 ms (lognormal, cv 0,5)
Fallas	Caída del servidor de 10 min en el minuto 20 (variante)
Réplicas	10 (semillas 1 a 10); 5 por variante de sensibilidad

Tabla 6. D1 antes y después de T2 (peor de 10 réplicas; fitness/resultados/d1.md).

Condición	Antes	Después (T2)	¿Cumple D1 después?
Supuestos base (2 núcleos)	100,00 %	100,00 % (máximo de 298 s a 18 s)	Sí
Caída de 10 min en el minuto 20	**97,47 %**	100,00 %	Sí
1 núcleo, como el droplet (variante, 5 réplicas)	**98,82 %**	100,00 %	Sí
1 núcleo y caída de 10 min	**90,22 %**	99,86 %	Sí (margen de 0,36 puntos)
Filas duplicadas	0	0	Sí



Figura 2. D1 con una caída de 10 min: conexiones en el proxy y p95 hasta guardar, antes y después de T2 (add/img/d1-caida-10min.svg).

4. **Registre en add/iteracion-1.md: objetivo, elemento refinado, conceptos evaluados, decisión, evidencia (gráfico o tabla) y si el escenario se cumple. Commit: E2: iteracion 1 ADD.**

`docs/architecture/add/iteracion-1.md` tiene las 12 secciones: entradas, objetivo, línea base, estimaciones, tácticas, trade-offs, rechazos, decisión, evidencia (modelo, supuestos, resultados, sensibilidad y la corrección a 1 núcleo), cumplimiento de D1, riesgos y lista de verificación. **D1 se cumple en la simulación con T2**; la parte "0 reportes perdidos en el dispositivo" queda sin evidencia (RI-05). Commit `a0f35d7` "E2: Iteracion 1 ADD".

E3: ATAM-lite cruzado

1. **El docente asigna a cada grupo el repositorio de otro grupo.**
2. **Revisen su árbol de utilidad e iteración 1. Usen el Prompt IA 3 como apoyo, pero verifiquen cada hallazgo contra la evidencia del grupo evaluado.**
3. **Abran un issue en el repositorio evaluado con al menos: 1 punto de sensibilidad, 1 trade-off, 2 riesgos (con probabilidad e impacto) y 1 no riesgo.**
4. **En los últimos 15 minutos, respondan el issue que recibieron y cópienlo a atam-lite.md, indicando si aceptan cada hallazgo o lo refutan con evidencia.**

No se realizó. El laboratorio se desarrolló de forma individual y no hubo otro grupo con quien intercambiar la evaluación: no se abrió ni se recibió ningún issue. `atam-lite.md` lo declara y registra solo hallazgos propios (sección 1), sin inventar hallazgos externos. Como sustituto parcial, la evaluación propia de E7 aplica el mismo formato (puntos de sensibilidad, trade-offs, riesgos con probabilidad e impacto y no riesgos) a las tres iteraciones.

E4: Iteraciones 2 y 3 de ADD

1. **Realicen dos iteraciones más, cada una con un escenario (H,*) distinto. Al menos una debe atender disponibilidad o seguridad.**
2. **Cada iteración sigue los 7 pasos de ADD y produce evidencia; se acepta una comparación de alternativas cuando el atributo no se puede simular (por ejemplo, seguridad), como en el Paso 6.**
3. **Si la evaluación cruzada (E3) detectó un riesgo crítico, una de estas iteraciones debe mitigarlo.**

Tabla 7. Iteraciones 2 y 3 (add/iteracion-2.md y add/iteracion-3.md).

	Iteración 2	**Iteración 3**
Escenario	**R2** (H,H), rendimiento	**S2** (H,M), seguridad
Elementos refinados	Módulo y API del mapa, panel del coordinador, proxy inverso y PostgreSQL	API de ingesta, módulo de priorización, PWA (identificación) y eventos al operador; nuevo componente Dispositivos
Conceptos evaluados	T5 agrupar por zona + gzip; T6 resumen del worker; T7 caché compartida; T8 réplica de lectura	T9 identificador de dispositivo; T10 límite por IP; T11 marcar y un voto por dispositivo; T12 código por SMS
Decisión	T5 aceptada; T6, T7 y T8 rechazadas	T9 y T11 aceptadas; T10 y T12 rechazadas
Evidencia	Prototipo `fitness/prototipo_r2.py`: consultas reales sobre 20 000 reportes en SQLite y modelo de red	Simulación por conteo `fitness/simulacion_s2.py` (10 réplicas) + comparación de alternativas para T12, que no se puede simular
Resultado	Carga del mapa p95: de **13,46 s a 1,00 s**; respuesta de 826 KB a 28 KB; filtro 0,68 s; retraso 31 s	Falsos aceptados: de **1 000 a 10**; zonas alteradas: de 5 a 0; legítimos: 100 %; aviso en 4 s
¿Cumple?	**Sí**	**Sí**, salvo un ataque Sybil (riesgo residual aceptado, R-10)
ADR	ADR-007	ADR-008

Tabla 8. Evidencia de S2: el límite por IP falla por el CGNAT (peor de 10 réplicas; fitness/resultados/s2.md).

Táctica	**Máx. aceptados por dispositivo (≤ 10)**	**Zonas alteradas (0)**	**Legítimos (≥ 99,5 %)**	**Aviso (≤ 300 s)**	**¿Cumple?**
Antes: sin control	1 000	5	100,00 %	sin aviso	No
T10: límite por IP	10	12	**36,69 %**	3,4 s	No
T9: límite por dispositivo	10	0	100,00 %	4,0 s	Sí
T9 + T11	10	0	100,00 %	4,0 s	Sí

Al menos una iteración atiende seguridad (S2), y la 1 ya atendió disponibilidad. No hubo riesgos de E3 que mitigar, porque E3 no se realizó. Commit `f514a26` "E4: Iteraciones 2 y 3".

E5: Fitness functions en integración continua

1. **Escriban 2 o más fitness functions estructurales que verifiquen las reglas de su ADR-001 y ADR-005 (import-linter en Python o ArchUnit en Java).**

Tabla 9. Contratos de .importlinter (4 kept, 0 broken).

Contrato	**Tipo**	**Regla**	**ADR**
`adr001-ingesta-sin-ia`	forbidden	Entrada, servicio, fábrica, adaptadores, decoradores, puertos, modelo, estados, eventos y observadores no importan `modulo.ia`	ADR-001
`adr001-ia-aislada`	forbidden	`modulo.ia` no importa ningún módulo del flujo de reportes	ADR-001
`adr005-capas`	layers	entrada > servicio y observadores > fábrica > adaptadores y decoradores > puertos, estados y eventos > modelo	ADR-005
`adr005-servicio-sin-`	forbidden	Servicio, dominio y puertos de IA no importan `externos`,	ADR-005

`infraestructura`		`adaptadores` ni `ia.adaptador_llm`	

Al escribir el cuarto contrato, este falló sobre el código real: `servicio.py` importaba `externos` y los adaptadores, en contra de ADR-005. Se corrigió el código, no el contrato: el endpoint HTTP, el controlador y la demostración pasaron a `entrada.py` (commit `c234e7f`).

2. Comprueben que funcionan: introduzcan una violación en una rama, verifiquen que falla, documenten la salida y reviértanla.

Rama `prueba-violacion-fitness`: el commit `3fd719e` agrega dos imports prohibidos y `117b86b` los revierte. Salida de `PYTHONPATH=src:src/modulo lint-imports` sobre `3fd719e` (código de salida 1):

```
+ src/modulo/servicio.py: from .adaptadores import AdaptadorIndexedDB
+ src/modulo/entrada.py:  from .ia.puerto import ClasificadorDanos


ADR-001: la ingesta solo valida, guarda y responde; no llama al clasificador de IA BROKEN
ADR-001: el componente de IA es un módulo con límites propios; ... KEPT
ADR-005: las dependencias van en un solo sentido (...) KEPT
ADR-005: el servicio y el dominio dependen de puertos, no de adaptadores ... BROKEN
Contracts: 2 kept, 2 broken.
modulo.entrada is not allowed to import modulo.ia:
-   modulo.entrada -> modulo.ia.puerto (l.15)
modulo.servicio is not allowed to import modulo.adaptadores:
-   modulo.servicio -> modulo.adaptadores (l.21)


# sobre 117b86b (revertido)
Contracts: 4 kept, 0 broken.
```

El contrato de capas no detecta `servicio → adaptadores` porque va hacia abajo; por eso hace falta `adr005-servicio-sin-infraestructura`. Detalle en `docs/architecture/fitness.md`.

3. Escriban 2 o más fitness functions de calidad que conviertan escenarios en pruebas (presupuesto de p95, 0 operaciones perdidas, aislamiento entre organizaciones, etc.).

En la entrega `tests/test_fitness.py` estaba vacío (0 bytes). En `90a3b13` tiene 8 pruebas, que corren en ≈ 5 s; cada una verifica también que la versión sin la táctica **no** cumple:

Tabla 10. Fitness functions de calidad (tests/test_fitness.py).

Prueba	Escenario y presupuesto	Qué ejecuta
`test_d1_modo_pico_guarda_99_5_por_ciento_en_15_min_sin_duplicados`	D1: ≥ 99,5 % ≤ 15 min; 0 duplicados	`simulacion_d1.py` con 1 núcleo y caída de 10 min (2 réplicas)
`test_r2_mapa_agrupado_por_zona_cumple_el_presupuesto_de_p95`	R2: carga p95 ≤ 3 s; filtro ≤ 1 s; retraso ≤ 60 s	`prototipo_r2.py`: consultas medidas en el runner
`test_s2_limite_y_voto_por_dispositivo_cumplen_el_escenario`	S2: ≤ 10 por dispositivo; 0 zonas; ≥ 99,5 % legítimos; aviso ≤ 300 s	`simulacion_s2.py` (3 réplicas)
`test_ingesta_no_pierde_ni_duplica_reportes_con_respuestas_perdidas`	0 operaciones perdidas: 50 de 50 enviados y 50 filas	Código real de `entrada.py` con 30 % de respuestas perdidas
`test_d1_sin_modo_pico_no_cumple`, `test_r2_listado_..._no_cumple`, `test_s2_sin_control_o_con_limite_por_ip_no_cumple` (2)	Control: la versión sin la táctica falla	Las mismas evidencias

Tabla 11. Mutaciones: cada fitness function de calidad falla si se quita su táctica.

Mutación en una copia	Resultado de test_fitness.py
Quitar `ON CONFLICT (id) DO NOTHING` en `AdaptadorPostgres`	1 falla (0 operaciones perdidas)
Quitar el modo pico (`DECISION = "Antes"`)	1 falla (D1)
Volver al listado de reportes en el mapa	1 falla (R2)
Límite por dispositivo de 50 en lugar de 10	1 falla (S2)

4. Configuren .github/workflows/fitness.yml para que se ejecuten en cada Pull Request; la última ejecución debe estar en verde.

```
name: fitness
on:
  pull_request:
  push:
    branches: [main]
  workflow_dispatch:
jobs:
  fitness:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: python -m pip install pytest==8.3.3 import-linter==2.1
      - name: Fitness functions estructurales (import-linter)
        env: { PYTHONPATH: src:src/modulo }
        run: lint-imports
      - name: Fitness functions de calidad y pruebas
        run: python -m pytest -v
```

En la entrega el archivo estaba vacío. Reproducido en un entorno limpio con las mismas versiones: `Contracts: 4 kept, 0 broken.` y `99 passed` (caracterización, patrones del Lab 07 y fitness).

E6: ADR y modelo C4 actualizado

1. Redacten un ADR por iteración (ADR-006 a ADR-008) con la plantilla del Lab 04, incluyendo la evidencia y el trade-off aceptado.

Tabla 12. ADR del Lab 08 (docs/architecture/adr/), con la plantilla del Lab 04: Estado, Fecha, Decisores, Contexto, Alternativas, Decisión y Consecuencias.

ADR	Decisión	Evidencia	Trade-off aceptado
006: Modo pico en la ingesta	Datos primero y fotos diferidas al pasar el 70 % de conexiones	D1 de 90,22 % a 99,86 % con 1 núcleo y caída de 10 min	Se difiere el 69 % de las fotos (13 723 de 20 000); la priorización sin foto es más fácil de engañar (S2)
007: API del mapa agregada por zona	Un registro por zona con GROUP BY y gzip en el proxy	Carga p95 de 13,46 s a 1,00 s	La vista nacional pierde el detalle por reporte; el filtro deja de ser local (0,68 s)
008: Identificador de dispositivo	Identificador firmado (HMAC), límite de 10 en 10 min y un voto por zona	Falsos de 1 000 a 10; 0 zonas alteradas; 100 % legítimos	Un dato personal más (Ley 29733); un celular con 3 daños pesa como un voto; Sybil no se frena

2. Actualicen workspace.dsl (Lab 06): nuevos contenedores o tecnologías y, en la descripción de cada elemento, el escenario que atiende [QA-xx]. Ejecuten structurizr validate y exporten la vista de contenedores.

No hay contenedores nuevos: las tres decisiones caben en los existentes (ADR-001). Hay un componente nuevo (Dispositivos, ADR-008) y dos tecnologías nuevas: gzip en el proxy (ADR-007) y HMAC-SHA256 del módulo `hmac` de Python (ADR-008). Ejemplos de las etiquetas:

```
api = container "API" "Monolito modular: ingesta idempotente con modo pico [QA-D1], mapa agregado
    por zona [QA-R2], control de roles [QA-S1], límite por dispositivo [QA-S2], ..." "Python"
dispositivos = component "Dispositivos" "Emite identificadores de dispositivo firmados, cuenta los
    reportes de cada dispositivo en 10 min ... (ADR-008) [QA-S2]." "Python, HMAC-SHA256"
proxy = infrastructureNode "Proxy inverso" "Termina TLS 1.2 o superior [QA-S1], ... comprime las
    respuestas JSON (ADR-007) [QA-R2] y expone la ocupación de conexiones ... (ADR-006) [QA-D1]."


$ structurizr.sh validate -w docs/architecture/c4/workspace.dsl      # código de salida 0
$ structurizr.sh export -w ... -f plantuml/c4plantuml -o export     # y -f mermaid
```

En la revisión, `verificar_c4.py` (Lab 06) encontró que el componente Dispositivos no estaba en `paquetes.puml`; se agregó el paquete con sus dos dependencias y la comprobación volvió a 0 fallas.

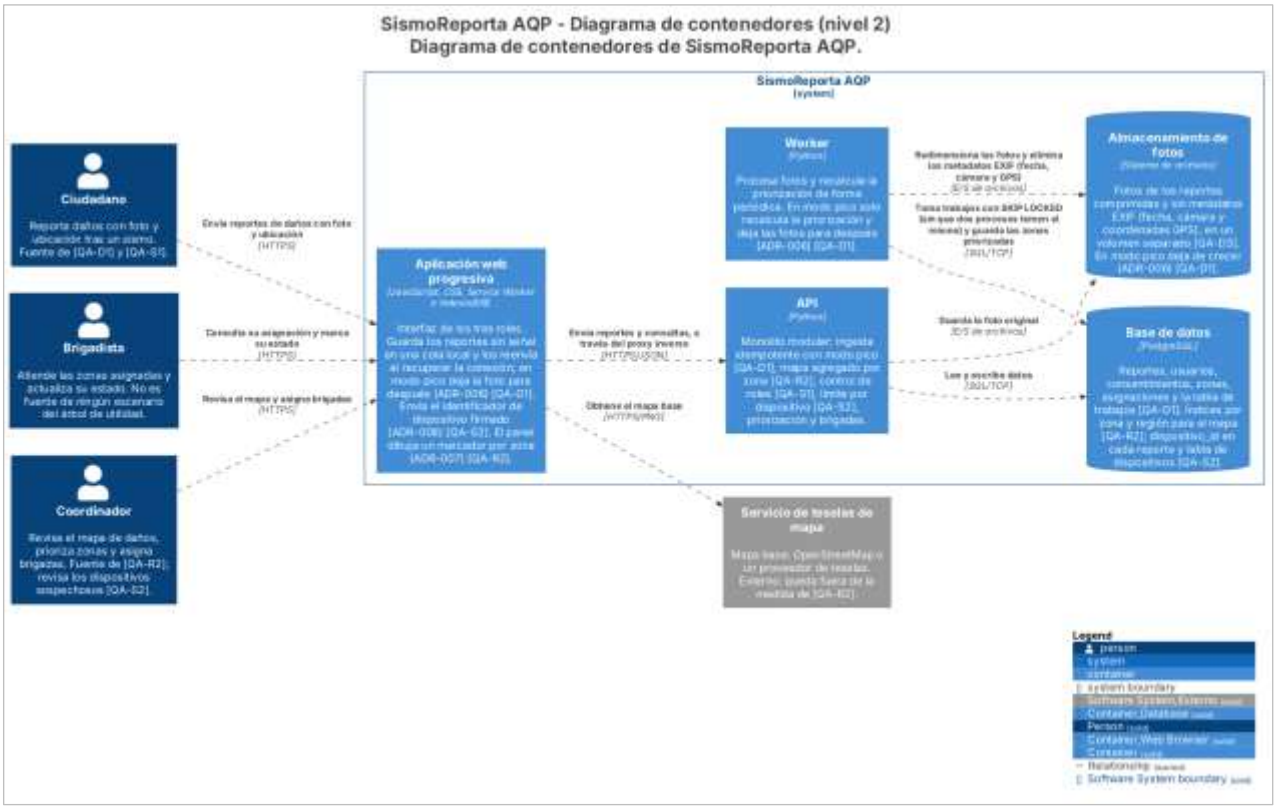


Figura 3. Vista de contenedores exportada, con las etiquetas [QA-xx] (c4/img/structurizr-Contenedores.png).

E7: Informe ATAM-lite y bitácora

1. **Completen atam-lite.md con todos los hallazgos (propios y recibidos) y una matriz de probabilidad e impacto (puede adaptar el script del ejemplo).**

`docs/architecture/atam-lite.md`: 15 riesgos, 7 puntos de sensibilidad, 5 puntos de trade-off y 6 no riesgos, todos propios (no hubo hallazgos recibidos, ver E3). La matriz la genera `docs/architecture/riesgos/matriz_riesgos.py` (escala 1–3; zona crítica: P × I ≥ 6).

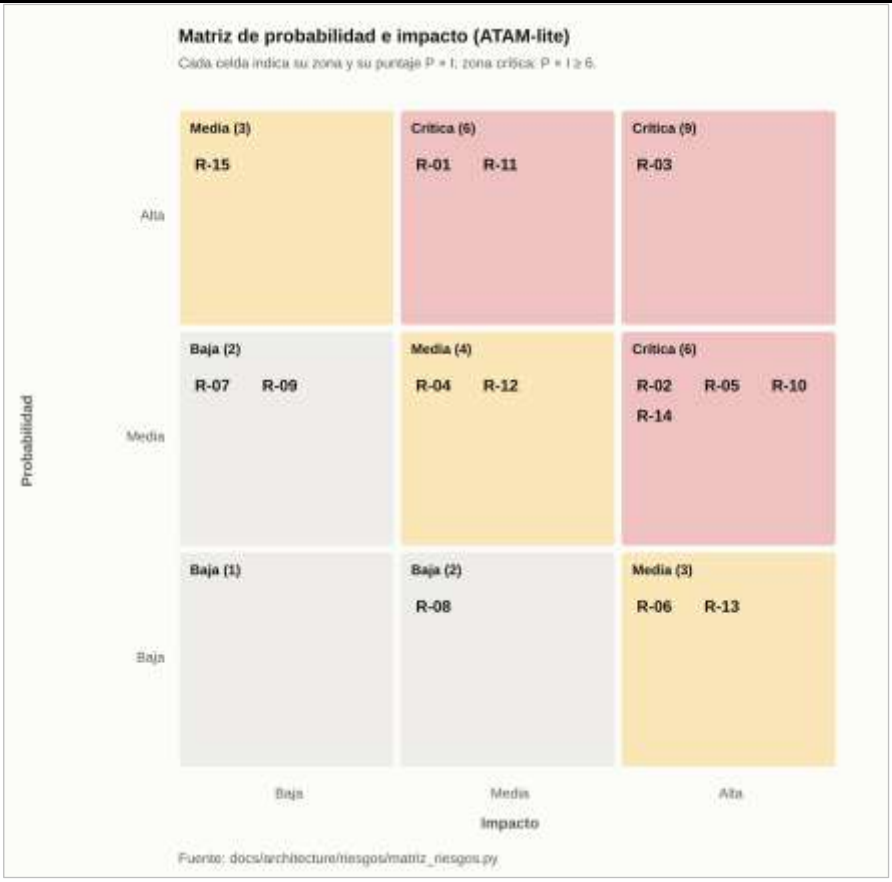


Figura 4. Matriz de probabilidad e impacto (riesgos/matriz-riesgos.svg).

2. Para cada riesgo de la zona crítica, indiquen la mitigación o la razón para aceptarlo.

Tabla 13. Riesgos de la zona crítica y respuesta.

ID	Riesgo	P × I	Respuesta
R-03	La simulación de D1 suponía 2 núcleos; el droplet tiene 1 vCPU	9	**Mitigado:** se repitió con 1 núcleo; T2 sigue cumpliendo (99,86 % con caída)
R-01	Fotos diferidas que nunca llegan	6	**Mitigado en parte y aceptado:** la PWA muestra y reintenta cada foto; D1 mide el reporte, que es lo que mueve a las brigadas
R-02	Evidencia de modelos, sin prueba de carga real	6	**Planificado:** prueba de carga con Locust y 1 vCPU
R-05	0 reportes perdidos en el celular, sin evidencia	6	**Planificado:** `navigator.storage.persist()` y pruebas en celulares de gama baja
R-10	Ataque Sybil: muchas identidades de dispositivo	6	**Aceptado:** la única táctica que lo frena (SMS) rompe RF-01 y cuesta de 13 a 33 veces R-03
R-11	El identificador de dispositivo es un dato personal	6	**Mitigado:** seudónimo, incluido en el consentimiento y con la retención de S3
R-14	D2 y D4 no tienen iteración propia	6	**Planificado:** la próxima iteración atiende D2

3. Completen bitacora-ia.md con 5 o más interacciones, incluyendo al menos un falso positivo de la IA refutado con evidencia.

`docs/architecture/bitacora-ia.md` tiene 8 interacciones del Lab 08 (de la 7 a la 14):

Tabla 14. Interacciones del Lab 08 en la bitácora.

#	Ejercicio	Verificación o corrección destacada
7	E1: escenarios	Contexto de otro caso detectado y reemplazado; **falso positivo** (la búsqueda de "rápido" marcó la propia lista de verificación); R2 corregido a (H,H)

8	E2: tácticas	T3 pasó de "margen útil" a rechazada al reanalizar
9	E2: evidencia	"Antes" ya cumplía con los supuestos base; no se forzaron los supuestos: se agregó la caída de 10 min
10	E4: iteraciones 2 y 3	**Falso positivo** en S2: las 12 zonas "alteradas" con T10 no las movía el ataque sino los legítimos bloqueados
11	E5: contratos	El contrato del servicio falló sobre el código real; se separó `entrada.py`
12	E6: ADR y C4	La simulación suponía 2 núcleos y el droplet tiene 1 (R-03)
13	E7: ATAM-lite	No se inventaron hallazgos recibidos
14	Revisión (2026-11-13)	Workflow y `test_fitness.py` vacíos; paquete Dispositivos faltante

Commit `0b802c3` (E6 y E7 entraron junto con E8); la interacción 14 y la sección 7 de `atam-lite.md`, en `05593f1`.

E8: README, revisión y entrega

1. **Agreguen al README.md la sección "Diseño de la arquitectura (Lab 08)": árbol de utilidad, tabla de iteraciones (escenario, táctica, evidencia, ¿cumple?), enlace a los ADR y la insignia (badge) del workflow de fitness functions.**

Tabla 15. Contenido de la sección del README en v0.8-arquitectura.

Elemento pedido	En la entrega (0b802c3)	En v0.8-arquitectura
Árbol de utilidad	Sí (`arbol.png` y enlace a `escenarios.md`)	Sí
Tabla de iteraciones	Sí (D1, R2 y S2, con táctica, evidencia y ¿cumple?)	Sí
Enlace a los ADR	Sí (ADR-006 a ADR-008)	Sí
Insignia del workflow	**No** (el workflow estaba vacío)	Sí, enlazada a `actions/workflows/fitness.yml`
Fitness functions	Solo una mención a `.importlinter`	Tabla de las estructurales y de calidad, y enlace a `fitness.md`

```
## Diseño de la arquitectura (Lab 08)
[![fitness](https://github.com/DavidOliva06/cs-2026b-lab04-grupo06/actions/workflows/fitness.yml/badge.svg)]
(https://github.com/DavidOliva06/cs-2026b-lab04-grupo06/actions/workflows/fitness.yml)
```

Los 98 enlaces relativos del README apuntan a archivos que existen.

2. **Cada integrante abre un Pull Request con uno de los entregables; el workflow debe pasar antes de aprobarlo.**

Integrante único: un Pull Request de `lab08-correcciones` hacia `main`. El workflow se dispara en cada Pull Request; su ejecución en GitHub depende de publicar la rama (el resultado esperado, reproducido en local, es 4 contratos cumplidos y 99 pruebas aprobadas). La rama `prueba-violacion-fitness` sirve para mostrar el workflow en rojo sobre `3fd719e` y en verde tras `117b86b`.

3. **Creen la etiqueta v0.8-arquitectura y entreguen el enlace del repositorio en el Aula Virtual hasta el sábado 31/10/2026 a las 23:59.**

El 31/10/26 no se creó la etiqueta. Se creó el 2026-11-13 sobre el commit corregido; su mensaje indica la fecha para no confundirla con la entrega original:

```
git tag -a v0.8-arquitectura -m "Lab 08: diseño guiado por atributos de calidad, con las
    correcciones de la revisión (2026-11-13)"
git push origin lab08-correcciones prueba-violacion-fitness v0.8-arquitectura


$ git log --oneline -4 v0.8-arquitectura
05593f1 Lab 08: correcciones tras la revisión del informe
```

```
6479ce9 Incorpora la prueba de violación de las fitness functions (E5.2)
117b86b Revierte la violación de prueba de las fitness functions
3fd719e Prueba de las fitness functions: violación intencional de ADR-001 y ADR-005
```

Revisión de la entrega

Tabla 16. Problemas de la entrega del 31/10/26 y solución.

#	Problema	Evidencia	Solución
1	`.github/workflows/fitness.yml` vacío: no había CI ni insignia (E5.4, E8.1)	0 bytes desde `9a28bdb`	Workflow con import-linter y pytest (`90a3b13`); insignia en el README
2	`tests/test_fitness.py` vacío: sin fitness functions de calidad (E5.3)	0 bytes	8 pruebas de D1, R2, S2 y 0 operaciones perdidas, con mutaciones
3	La violación se probó en una copia local, no en una rama (E5.2)	Bitácora, interacción 11	Rama `prueba-violacion-fitness` con la salida documentada
4	Sin etiqueta de entrega (E8.3)	`git ls-remote --tags`: ninguna `v0.8`	`v0.8-arquitectura` sobre `05593f1`
5	`paquetes.puml` sin el módulo de ADR-008	`verificar_c4.py`: 2 fallas	Paquete `dispositivos` agregado; 0 fallas
6	La rama del Lab 08 no tenía las correcciones del Lab 07 (25 pruebas en lugar de 91)	`pytest` sobre `0b802c3`	Fusión con `v0.7-patrones`, uniendo `entrada.py` con el Observer
7	Informe: prompt de E1 con el contexto del ejemplo de la guía (otro caso), cuestionario del Lab 05 y fecha 10/10/26	Informe entregado	Prompt con el contexto de SismoReporta, cuestionario del Lab 08 y fecha 31/10/26

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿En qué se diferencia diseñar la arquitectura con ADD de elegir un estilo arquitectónico como en el Laboratorio 04?

En el Lab 04 comparamos tres estilos completos y elegimos uno (monolito modular con ingesta asíncrona, ADR-001) con una matriz de criterios. ADD parte de ese estilo y lo refina por iteraciones: en cada una toma un escenario con medida (D1, R2, S2), elige los elementos a refinar, evalúa tácticas concretas y decide con evidencia. Ninguna de las tres iteraciones cambió el estilo; las tácticas se implementaron dentro de los contenedores que ya existían.

2. ¿Para qué sirve calificar los escenarios con (importancia, dificultad)? ¿Qué escenario de su árbol tiene la mayor prioridad y por qué?

Sirve para decidir el orden de las iteraciones: primero lo que más importa y más riesgo tiene, y para dejar fuera lo que no vale el esfuerzo (R3, S3 y M3). La mayor prioridad es D1 (H,H): la disponibilidad es el atributo crítico, un reporte perdido retrasa la ayuda, y es difícil porque la red congestionada y el borrado de datos del celular están fuera del control del servidor.

3. ¿Qué diferencia hay entre una táctica y un patrón? Ilustre con el circuit breaker.

Una táctica es una decisión de diseño que afecta un solo atributo de calidad; un patrón es una solución empaquetada que combina varias tácticas y fija su estructura. Por ejemplo, "detectar una falla" y "dejar de llamar a un servicio que falla" son tácticas de disponibilidad; el circuit breaker es el patrón que las junta con estados (cerrado, abierto y semiabierto), un umbral de fallas y un tiempo de espera. En nuestro caso, el modo pico (T2) es una táctica de degradación.

4. Describa un punto de sensibilidad y un punto de trade-off de su arquitectura, con la evidencia que los respalda.

Punto de sensibilidad: los núcleos del servidor. Con 2 núcleos, la arquitectura "antes" cumple D1; con 1, como el droplet real, baja a 98,82 % y no cumple (PS-2). Punto de trade-off: el modo pico de ADR-006 mejora la disponibilidad (D1 de 90,22 % a 99,86 % con 1 núcleo y caída de 10 min), pero empeora la integridad de la priorización, porque sin foto es más fácil engañarla, y en la simulación base se difiere el 69 % de las fotos (PT-1).

5. ¿Qué supuestos hizo su simulación y cómo podrían invalidar la decisión si fueran falsos?

Supusimos una subida de 50 kbps, fotos de 300 KB, 1 000 conexiones en el proxy, 30 ms por reporte y reintentos con espera de hasta 120 s. Si las fotos fueran más pequeñas o la red más rápida, el modo pico casi no se activaría y no haría falta. Si el servidor procesara más lento de lo supuesto o las caídas duraran más de 10 a 12 minutos, ni el modo pico alcanzaría: con una caída de 13 min no se cumple D1. Además, la simulación no modela el borrado de datos en el celular.

6. ¿Qué riesgo de la evaluación cruzada aceptaron sin mitigarlo y por qué es razonable para un plazo de 1 mes?

No recibimos riesgos de otro grupo, porque la evaluación cruzada no se realizó (trabajo individual). De nuestra propia evaluación aceptamos sin mitigar el ataque Sybil (R-10): un atacante con muchas identidades de dispositivo. Es razonable porque la única táctica que lo frena, el código por SMS, rompe el reporte sin señal, cuesta de 13 a 33 veces el presupuesto mensual por simulacro y necesita un proveedor externo, algo imposible de integrar y probar en un mes.

7. Explique qué es una fitness function y qué degradación concreta de su arquitectura detectaría cada una de las suyas.

Es una prueba automática que mide si la arquitectura sigue cumpliendo una característica. Los contratos de import-linter detectan que la ingesta llame a la IA, que la IA dependa del flujo, que las capas se inviertan o que el servicio vuelva a importar adaptadores. Las de calidad detectan que se quite el modo pico (D1 bajaría de 99,5 %), que el mapa vuelva al listado de reportes (p95 sobre 3 s), que se afloje el límite por dispositivo (S2) o que la ingesta pierda la protección contra duplicados.

8. ¿Qué propuesta de la IA rechazaron con evidencia? ¿Qué habría pasado si la aceptaban sin medir?

La IA recomendó T1 (limitar la respuesta con HTTP 429) junto con el modo pico. La simulación mostró que T1 no mejoraba D1 en ninguna variante y que, con 1 núcleo y una caída de 10 min, T1 + T2 no cumplía (99,28 %) mientras que solo T2 sí (99,86 %). Si la aceptábamos sin medir, habríamos agregado complejidad en el proxy y en la PWA que, además, empeoraba el escenario crítico.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] Bass, L., Clements, P. y Kazman, R. (2021). Software Architecture in Practice (4.ª ed.). Addison-Wesley.

[2] Cervantes, H. y Kazman, R. (2016). Designing Software Architectures: A Practical Approach. Addison-Wesley.

[3] Kazman, R., Klein, M. y Clements, P. (2000). ATAM: Method for Architecture Evaluation (CMU/SEI-2000-TR-004). Software Engineering Institute.

[4] Ford, N., Parsons, R., Kua, P. y Sadalage, P. (2022). Building Evolutionary Architectures (2.ª ed.). O'Reilly Media.

[5] Nygard, M. T. (2018). Release It! Design and Deploy Production-Ready Software (2.ª ed.). Pragmatic Bookshelf.

[6] import-linter. Documentación oficial. https://import-linter.readthedocs.io/

[7] GitHub. Adding a workflow status badge. https://docs.github.com/actions/monitoring-and-troubleshooting-workflows/adding-a-workflow-status-badge

[8] Oliva Valdivia, D. (2026). Repositorio cs-2026b-lab04-grupo06, etiqueta v0.8-arquitectura. https://github.com/DavidOliva06/cs-2026b-lab04-grupo06